Национальный исследовательский университет
«Высшая школа экономики»

EcoLab2

Включение и агрегирование компонентов калькулятора в компонент Eco.Lab1. Компонент Eco.Lab2

Лабораторная работа 2

Выполнил:
Боронин Фёдор Дмитриевич
группа 24ТИДИ

Преподаватель:
Башев Владимир Николаевич

2026

Содержание

1. Постановка задачи
2. Включение и агрегирование
3. Компоненты калькулятора A-E
4. Доработка Eco.Lab1
5. Компонент Eco.Lab2
6. Замена компонента A на C
7. Тестирование
8. Анализ
9. Сборка
10. Выводы
11. Источники

1. Постановка задачи

В ранее созданный компонент Eco.Lab1 (лабораторная работа 1, решение систем ОДУ) нужно добавить компоненты калькулятора, которые выполняют сложение, вычитание, умножение и деление. Для этого используются два приема технологии АОМК: включение и агрегирование. Калькуляторы берутся из учебного курса InsideACOM.

Требования задания:

- в клиенте показать запрос интерфейсов калькулятора через `QueryInterface` у интерфейса Eco.Lab1 и вызов их методов;
- показать свойства интерфейсов: через любой указатель на интерфейс компонента можно получить любой другой его интерфейс;
- на оценку 9-10: добавить в Eco.Lab1 поддержку агрегирования внешним компонентом, сделать новый компонент Eco.Lab2 и показать включение и агрегирование всех компонентов A, B, C, D, E и Eco.Lab1;
- предоставить исходный код, сборку, клиент в виде unit-теста и отчет с анализом и выводами.

Важное ограничение технологии: опубликованный интерфейс менять нельзя. Поэтому интерфейс `IEcoAdvancedMath` из первой работы остался прежним, а компонент расширен новыми интерфейсами и изменениями внутри реализации. Все 50 проверок первой работы проходят без изменений.

2. Включение и агрегирование

Технология АОМК не поддерживает наследование реализации, но его можно смоделировать, если один компонент использует другой. Для этого есть два приема.

Включение. Внешний компонент создает внутренний и хранит указатель на его интерфейс. Если внешний компонент сам предоставляет тот же интерфейс, он реализует его заново: в каждой функции своей таблицы вызывает функцию внутреннего компонента. До и после вызова можно добавить свой код. Клиент видит только интерфейс внешнего компонента и о внутреннем не знает.

Агрегирование. Внешний компонент отдает клиенту указатель на интерфейс внутреннего компонента напрямую, без своей обертки. Трудность в том, что у полученного интерфейса функции `QueryInterface`, `AddRef` и `Release` принадлежат внутреннему компоненту, а клиент должен видеть один компонент: с одним

счетчиком ссылок и с полным набором интерфейсов. Поэтому внутренний компонент должен поддерживать агрегирование:

- у него два варианта `IEcoUnknown`. Неделегирующий работает с его собственными данными. Делегирующий передает все вызовы управляющему `IEcoUnknown`;
- при создании ему передается указатель на `IEcoUnknown` внешнего компонента. Если компонент агрегирован, управляющим становится внешний `IEcoUnknown`, иначе собственный неделегирующий;
- внешний компонент хранит указатель на неделегирующий `IEcoUnknown` внутреннего и через него запрашивает интерфейсы и освобождает внутренний компонент.

Правила `QueryInterface`, которые должны выполняться для любого компонента:

- всегда возвращается один и тот же `IEcoUnknown`, через какой бы интерфейс его ни запрашивали;
- если интерфейс удалось получить один раз, его можно получить снова;
- можно запросить интерфейс, который уже есть;
- можно вернуться к интерфейсу, с которого начали;
- если интерфейс можно получить хоть откуда-нибудь, его можно получить откуда угодно.

Для агрегирования эти правила и являются проверкой правильности: интерфейс внутреннего компонента должен вести себя так же, как собственный интерфейс внешнего.

3. Компоненты калькулятора A-E

Калькуляторы взяты из учебного курса InsideACOM (репозиторий peerf-eco/Eco.Education, уроки 2-7). Они реализуют два интерфейса: `IEcoCalculatorX` (`Addition`, `Subtraction`) и `IEcoCalculatorY` (`Multiplication`, `Division`).

Таблица 1. Компоненты калькулятора

Компонент	Интерфейсы	Устройство	Агрегирование
A	X	простой компонент	нет
B	X	делегирующий и неделегирующий IEcoUnknown	поддерживает
C	X, Y	новая версия A с тем же CID	нет
D	X, Y	включает B (если B нет, то A)	нет
E	Y (и X через B)	агрегирует B	нет

Из таблицы видно, что агрегировать можно только B. Поэтому все остальные калькуляторы используются только через включение, а для агрегирования

используются B и Eco.Lab1, в который поддержка агрегирования добавлена в этой работе.

Чтобы исходные тексты курса собрались с текущей версией Eco.Framework (сентябрь 2026) и компилятором в режиме C89, в них сделаны две правки: константа `ERR_ECO_SUCCESES` из старой версии библиотеки заменена на `ERR_ECO_SUCCESS`, и удалена закомментированная строка с комментарием вида `//`, который не допускается стандартом C89. Логика компонентов не менялась. Для каждого калькулятора написаны файлы сборки под все платформы по образцу Eco.Lab1.

4. Доработка Eco.Lab1

Схема компонента после доработки показана на рисунке 1.

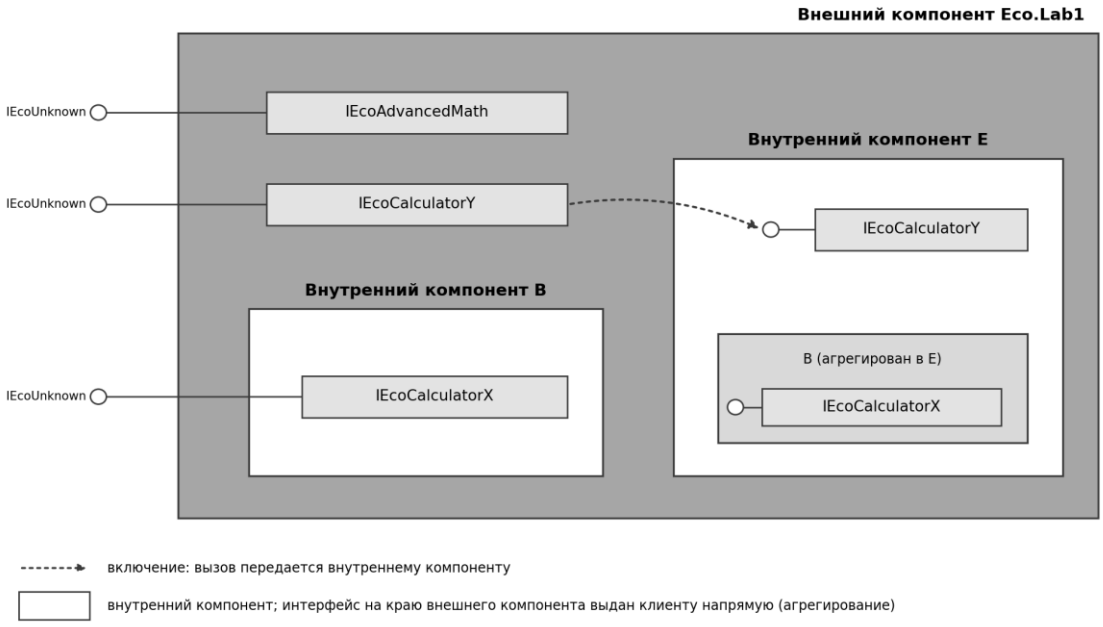


Рис. 1. Eco.Lab1: X агрегирован из B, Y включен из E

Структура экземпляра компонента. Добавлены таблица функций `IEcoCalculatorY`, таблица неделегирующего `IEcoUnknown`, указатель на управляющий `IEcoUnknown` и указатели на внутренние компоненты:

```
typedef struct CEcoLab1_847FF6CC {
    IEcoAdvancedMathVTbl* m_pVTblIEcoAdvancedMath;
    IEcoCalculatorYVTbl* m_pVTblIY;
    IEcoUnknownVTbl* m_pVTblINondelegatingUnk;
    ...
    uint32_t m_cRef;
    IEcoMemoryAllocator1* m_pIMem;
    IEcoSystem1* m_pISys;

    IEcoUnknown* m_pIUnkOuter;
    IEcoUnknown* m_pInnerUnknown;
    IEcoCalculatorY* m_pIY;
} CEcoLab1_847FF6CC;
```

Таблица `IEcoAdvancedMath` стоит первой, поэтому указатель на этот интерфейс совпадает с адресом экземпляра, и функции первой работы (`Solve`, `Step`) остались без изменений. Для двух других таблиц адрес экземпляра получается вычитанием смещения поля: один указатель для `m_pVTblIY` и два для `m_pVTblINondelegatingUnk`.

4.1. Поддержка агрегирования

При создании компонент запоминает управляющий `IEcoUnknown`: внешний, если его передали, иначе свой неделегирующий.

```
pCMe->m_pIUnkOuter = (IEcoUnknown*)&pCMe->m_pVTblINondelegatingUnk;
if (pIUnkOuter != 0) {
    pCMe->m_pIUnkOuter = pIUnkOuter;
}
```

Функции `QueryInterface`, `AddRef` и `Release` у интерфейсов `IEcoAdvancedMath` и `IEcoCalculatorY` стали делегирующими: они ничего не делают сами и передают вызов управляющему `IEcoUnknown`.

```
static int16_t ECOCALLMETHOD CEcoLab1_847FF6CC_QueryInterface(IEcoAdvancedMathPtr_t
me, const UGUID* riid, void** ppv) {
    CEcoLab1_847FF6CC* pCMe = (CEcoLab1_847FF6CC*)me;

    if (me == 0 || ppv == 0) {
        return ERR_ECO_POINTER;
    }
    return pCMe->m_pIUnkOuter->pVTbl->QueryInterface(pCMe->m_pIUnkOuter, riid, ppv);
}
```

Вся логика выбора интерфейса перенесена в неделегирующий `QueryInterface`. Счетчик ссылок `m_cRef` теперь меняется только в неделегирующих `AddRef` и `Release`.

```
static int16_t ECOCALLMETHOD
CEcoLab1_847FF6CC_NondelegatingQueryInterface(IEcoUnknownPtr_t me, const UGUID* riid,
voidptr_t* ppv) {
    CEcoLab1_847FF6CC* pCMe = (CEcoLab1_847FF6CC*)((char_t*)me - 2 *
sizeof(voidptr_t));
    ...
    if ( IsEqualUGUID(riid, &IID_IEcoAdvancedMath) ) {
        *ppv = &pCMe->m_pVTblIEcoAdvancedMath;
        ((IEcoUnknown*)(*ppv))->pVTbl->AddRef((IEcoUnknown*)(*ppv));
    }
    else if ( IsEqualUGUID(riid, &IID_IEcoCalculatorY) && pCMe->m_pIY != 0 ) {
        *ppv = &pCMe->m_pVTblIY;
        ((IEcoUnknown*)(*ppv))->pVTbl->AddRef((IEcoUnknown*)(*ppv));
    }
    else if ( IsEqualUGUID(riid, &IID_IEcoCalculatorX) && pCMe->m_pInnerUnknown != 0 )
{
        return pCMe->m_pInnerUnknown->pVTbl->QueryInterface(pCMe->m_pInnerUnknown,
riid, ppv);
    }
    else if ( IsEqualUGUID(riid, &IID_IEcoUnknown) ) {
        *ppv = &pCMe->m_pVTblINondelegatingUnk;
        ++pCMe->m_cRef;
    }
    else {
        *ppv = 0;
        return ERR_ECO_NOINTERFACE;
    }
    return ERR_ECO_SUCCESS;
}
```

В фабрике компонента изменена одна строка: после создания экземпляра запрос интерфейса выполняется через неделегирующий `IEcoUnknown`. Иначе при агрегировании фабрика вызвала бы делегирующий `QueryInterface`, и запрос ушел бы внешнему компоненту, который в этот момент еще не готов.

```
pIUnk = (IEcoUnknown*)&pCObj->m_pVTblINondelegatingUnk;
result = pIUnk->pVTbl->QueryInterface(pIUnk, riid, ppv);
```

4.2. Интерфейс X: агрегирование компонента B

При инициализации Eco.Lab1 создает компонент B через интерфейсную шину, передает ему свой управляющий `IEcoUnknown` и запрашивает у B его неделегирующий `IEcoUnknown`. При агрегировании запрашивать можно только `IID_IEcoUnknown`.

```
pCMe->m_pInnerUnknown = 0;
if (pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoCalculatorB, pCMe->m_pIUnkOuter,
                                 &IID_IEcoUnknown, (void**) &pCMe->m_pInnerUnknown) !=
0) {
    pCMe->m_pInnerUnknown = 0;
}
```

На запрос `IID_IEcoCalculatorX` неделегирующий `QueryInterface` Eco.Lab1 передает вызов в B, и клиент получает указатель на таблицу функций самого компонента B. Своей реализации X у Eco.Lab1 нет. Когда клиент вызывает `AddRef`, `Release` или `QueryInterface` у этого X, компонент B передает вызов управляющему

IEcoUnknown, то есть обратно в Eco.Lab1. Так интерфейс B становится неотличим от собственного интерфейса Eco.Lab1.

Здесь передается именно m_pIUnkOuter, а не свой неделегирующий IEcoUnknown. Если Eco.Lab1 сам агрегирован в другой компонент, вызовы от B пойдут сразу самому внешнему компоненту. Это используется в Eco.Lab2 (раздел 5).

4.3. Интерфейс Y: включение компонента E

Для IEcoCalculatorY у Eco.Lab1 есть своя таблица функций. Внутренним компонентом служит E. Если E не зарегистрирован в шине, используется D: оба реализуют Y, и Eco.Lab1 все равно, кто из них работает внутри.

```
pCMe->m_pIY = 0;
if (pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoCalculatorE, 0, &IID_IEcoCalculatorY,
(void**) &pCMe->m_pIY) != 0
    || pCMe->m_pIY == 0) {
    pCMe->m_pIY = 0;
    if (pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoCalculatorD, 0,
&IID_IEcoCalculatorY, (void**) &pCMe->m_pIY) != 0) {
        pCMe->m_pIY = 0;
    }
}
```

Функции Multiplication и Division вызывают соответствующие функции внутреннего компонента. В Division добавлен свой код перед вызовом: проверка деления на ноль. Компонент E выполняет a / b без проверки, и деление на ноль привело бы к аварийному завершению программы. Это пример того, как при включении внешний компонент специализирует интерфейс внутреннего.

```
static int16_t ECOCALLMETHOD CEcoLab1_847FF6CC_IY_Division(IEcoCalculatorYPtr_t me,
int16_t a, int16_t b) {
    CEcoLab1_847FF6CC* pCMe = (CEcoLab1_847FF6CC*)((char_t*)me - sizeof(voidptr_t));

    if (me == 0 || pCMe->m_pIY == 0) {
        return -1;
    }
    if (b == 0) {
        return 0;
    }
    return pCMe->m_pIY->pVTbl->Division(pCMe->m_pIY, a, b);
}
```

При удалении экземпляра Eco.Lab1 освобождает включенный компонент через его интерфейс Y, а агрегированный B через неделегирующий IEcoUnknown.

Если калькуляторы не зарегистрированы в шине, Eco.Lab1 продолжает работать как в первой работе: запрос X или Y возвращает ERR_ECO_NOINTERFACE, а решение ОДУ доступно.

5. Компонент Eco.Lab2

Eco.Lab2 создан утилитой eco-wizard. Его схема показана на рисунке 2.

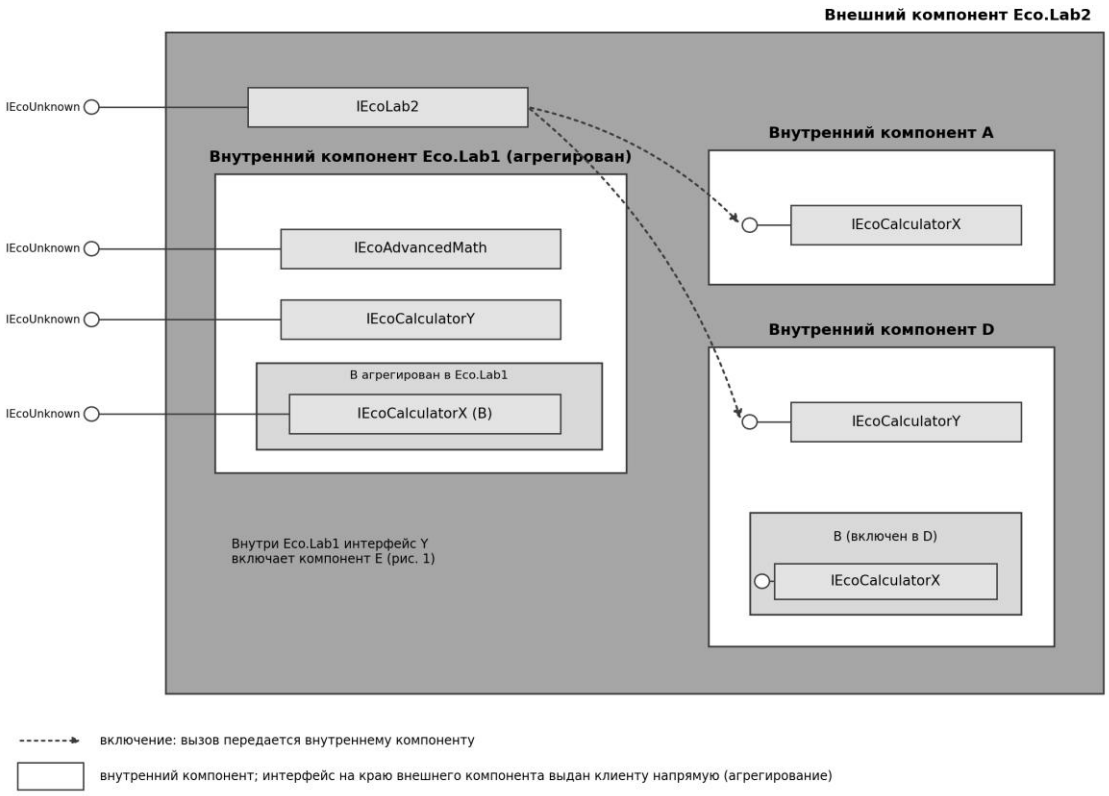


Рис. 2. Eco.Lab2: агрегирует Eco.Lab1, включает A и D

Собственный интерфейс компонента `IEcoLab2` содержит один метод: вычисление значения многочлена с целыми коэффициентами по схеме Горнера.

```
int16_t (ECOCALLMETHOD *Horner)(IEcoLab2Ptr_t me, const int16_t* coeffs, uint32_t n,
int16_t x, int32_t* result);
```

Eco.Lab2 использует оба приема:

- агрегирует Eco.Lab1. На запросы `IEcoAdvancedMath`, `IEcoCalculatorX` и `IEcoCalculatorY` он передает вызов неделегирующему `IEcoUnknown` компонента Eco.Lab1, и клиент получает интерфейсы Eco.Lab1 напрямую;
- включает A и D. В методе `Horner` Eco.Lab2 сам является клиентом внутренних компонентов: умножает через `IEcoCalculatorY` компонента D и складывает через `IEcoCalculatorX` компонента A.

```
else if ( (IsEqualUGUID(riid, &IID_IEcoAdvancedMath) || IsEqualUGUID(riid,
&IID_IEcoCalculatorX)
          || IsEqualUGUID(riid, &IID_IEcoCalculatorY)) && pCMe->m_pInnerUnknown != 0
) {
    return pCMe->m_pInnerUnknown->pVTbl->QueryInterface(pCMe->m_pInnerUnknown, riid,
ppv);
}
```

```
acc = coeffs[0];
for (i = 1; i < n; i++) {
    if (acc < -32768 || acc > 32767) {
        return ERR_ECO_FAIL;
    }
    t = pCMe->m_pIY->pVTbl->Multiplication(pCMe->m_pIY, (int16_t)acc, x);
    if (t < -32768 || t > 32767) {
        return ERR_ECO_FAIL;
    }
    acc = pCMe->m_pIX->pVTbl->Addition(pCMe->m_pIX, (int16_t)t, coeffs[i]);
}
*result = acc;
```

Методы калькулятора принимают аргументы типа `int16_t`, а возвращают `int32_t`. Если промежуточный результат не помещается в `int16_t`, передать его дальше нельзя, и метод возвращает `ERR_ECO_FAIL`, а не неверный ответ.

Интерфейс X, который клиент получает от Eco.Lab2, принадлежит компоненту B, и между ними два уровня агрегирования: B агрегирован в Eco.Lab1, Eco.Lab1 агрегирован в Eco.Lab2. Работает это потому, что Eco.Lab1 передал компоненту B не свой `IEcoUnknown`, а управляющий, которым при агрегировании является `IEcoUnknown` Eco.Lab2. Путь запроса интерфейса через такой X показан на рисунке 3.

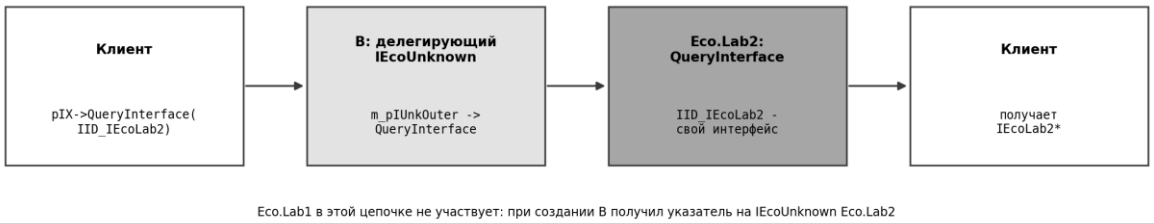


Рис. 3. Запрос IEcoLab2 через интерфейс X компонента B

Таблица 2. Как используются все компоненты

Компонент	Где используется	Прием
A	Eco.Lab2, сложение в Horner	включение
B	Eco.Lab1, интерфейс X	агрегирование
B	внутри D и внутри E	включение в D, агрегирование в E
C	вместо A в сборке EcoLab2C	замена компонента
D	Eco.Lab2, умножение в Horner	включение
E	Eco.Lab1, интерфейс Y	включение
Eco.Lab1	Eco.Lab2	агрегирование

6. Замена компонента A на C

Компоненты A и C имеют один и тот же идентификатор `CID_EcoCalculatorA`. C - это новая версия A: к интерфейсу X добавлен интерфейс Y. Две статические библиотеки с одинаковым CID нельзя собрать в одну программу, поэтому клиент

Eco.Lab2 собирается в двух вариантах. В файле сборки `MakefileExe` добавлена переменная `CALC_A`, которая указывает, из какой папки брать библиотеку:

```
make -f MakefileExe TARGET=1 DEBUG=0 ARCH=x86_64
make -f MakefileExe TARGET=1 DEBUG=0 ARCH=x86_64 OUT_NAME=EcoLab2C
CALC_A=Eco.CalculatorC
```

Код Eco.Lab2 и клиента в обоих вариантах один и тот же. Тест сам определяет, какой компонент получен: у A запрос `IEcoCalculatorY` возвращает `ERR_ECO_NOINTERFACE`, у C возвращает работающий интерфейс. Метод `Horner` в обоих случаях дает одинаковый результат. Так проверяется взаимозаменяемость компонентов: новую версию можно поставить вместо старой, и существующие клиенты продолжают работать.

7. Тестирование

Проверки находятся в двух клиентах. Клиент Eco.Lab1 (`EcoLab1`) содержит 50 проверок первой работы и 13 новых, всего 63. Клиент Eco.Lab2 (`EcoLab2` и `EcoLab2C`) содержит 20 проверок. Все проверки проходят.

Таблица 3. Проверки второй работы

Программа	Блок	Что проверяется	Проверок
EcoLab1	17	X и Y через QueryInterface у Eco.Lab1, арифметика, деление на ноль	4
EcoLab1	18	пять правил QueryInterface, отказ на чужой IID	6
EcoLab1	19	общий счетчик ссылок у IEcoAdvancedMath, X (из B) и Y	2
EcoLab1	20	A, B, D, E по отдельности, включение в D и агрегирование в E	1
EcoLab2	1	Horner: значения, ошибки, переполнение	5
EcoLab2	2	интерфейсы Eco.Lab1 через Eco.Lab2, решение ОДУ	5
EcoLab2	3	правила QueryInterface для четырех интерфейсов	4
EcoLab2	4	общий счетчик ссылок Eco.Lab2, Eco.Lab1 и B	2
EcoLab2	5	компонент A или C по одному CID	4

Запрос интерфейсов калькулятора у Eco.Lab1, как в задании:

```
r1 = pIEcoLab1->pVTbl->QueryInterface(pIEcoLab1, &IID_IEcoCalculatorX, (void**) &pIX);
r2 = pIEcoLab1->pVTbl->QueryInterface(pIEcoLab1, &IID_IEcoCalculatorY, (void**) &pIY);
c = pIX->pVTbl->Addition(pIX, 3, 5);
```

Вывод программы EcoLab1, блоки 17-20:

```
17. Калькулятор через Eco.Lab1: X агрегирован из B, Y включен из E
[OK] QueryInterface(IID_IEcoCalculatorX) и (IID_IEcoCalculatorY) у Eco.Lab1
  Addition(3, 5) = 8
  Subtraction(3, 5) = -2
  Multiplication(6, 7) = 42
  Division(20, 3) = 6
  Division(5, 0) = 0
[OK] сложение и вычитание
[OK] умножение и деление
[OK] деление на ноль перехвачено во внешнем компоненте

18. Правила QueryInterface
  IEcoUnknown через IEcoAdvancedMath 0x245223f0, через X 0x245223f0, через Y
0x245223f0
[OK] 1. всегда один и тот же IEcoUnknown
[OK] 2. интерфейс можно получить снова
[OK] 3. можно получить интерфейс, который уже есть (X из X)
[OK] 4. можно вернуться туда, откуда начали (X -> Y -> X)
[OK] 5. из любого интерфейса получается любой (Math -> X -> Y -> Math)
[OK] неподдерживаемый интерфейс: ERR_ECO_NOINTERFACE и нулевой указатель

19. Подсчет ссылок: все интерфейсы Eco.Lab1 работают с одним счетчиком
  AddRef: IEcoAdvancedMath 2, X (интерфейс компонента B) 3, Y 5
[OK] AddRef у агрегированного X увеличивает счетчик Eco.Lab1
  после Release всех полученных указателей: 1
[OK] после Release счетчик вернулся к исходному

20. Внутренние компоненты напрямую через интерфейсную шину
  A: 3 + 5 = 8
  B: 3 - 5 = -2
  D: 6 * 7 = 42, через включенный B: 3 + 5 = 8
  E: 20 / 3 = 6, через агрегированный B: 3 + 5 = 8
[OK] A, B, D, E работают и освобождаются (счетчик 0 после Release)

Итого: пройдено 63, провалено 0
```

Блок 19 главный для проверки агрегирования. Вызов `AddRef` у интерфейса X, который на самом деле принадлежит компоненту B, увеличил счетчик ссылок Eco.Lab1 с 2 до 3. Значит, B передает вызовы управляющему `IEcoUnknown`, как и должен.

Вывод программы EcoLab2, блоки 1-4:

```
1. IEcoLab2::Horner: умножение через включенный D, сложение через включенный A
p(x) = 2x^3 - 3x^2 + 5
     x       Horner напрямую
    -3          -76          -76
    -1            0            0
     2            9            9
     4           85           85
[OK] значения многочлена совпадают с прямым вычислением
[OK] корень x^2 - 1 в точке 1 дает 0
[OK] переполнение int16 в промежуточном результате -> ERR_ECO_FAIL
[OK] n = 0 -> ERR_ECO_INVALIDARG
[OK] coeffs = NULL -> ERR_ECO_POINTER

2. Eco.Lab1 агрегирован в Eco.Lab2: его интерфейсы отдаются клиенту напрямую
[OK] QueryInterface(IID_IEcoAdvancedMath) у Eco.Lab2
Solve (Рунге-Кутта 4) через Eco.Lab2: x(10) = -0.839071529524, cos(10) = -
0.839071529076
[OK] метод Рунге-Кутты работает через агрегированный Eco.Lab1
[OK] QueryInterface(IID_IEcoCalculatorX): X компонента B через два уровня
агрегирования
[OK] QueryInterface(IID_IEcoCalculatorY): Y из Eco.Lab1 (включен E)
  Addition(3, 5) = 8, Subtraction(3, 5) = -2, Multiplication(6, 7) = 42, Division(20,
3) = 6
[OK] арифметика через Eco.Lab2

3. Правила QueryInterface для четырех интерфейсов Eco.Lab2
IEcoUnknown через IEcoLab2 0x326513e0, Math 0x326513e0, X 0x326513e0, Y 0x326513e0
[OK] 1. один и тот же IEcoUnknown у всех интерфейсов, включая интерфейс B
[OK] из X компонента B получается IEcoLab2 внешнего компонента
[OK] 4. можно вернуться туда, откуда начали (Math -> X -> Y -> Math)
[OK] неподдерживаемый интерфейс: ERR_ECO_NOINTERFACE

4. Подсчет ссылок: интерфейсы агрегированных компонентов увеличивают счетчик Eco.Lab2
AddRef: IEcoLab2 2, X (компонент B) 3, IEcoAdvancedMath (Eco.Lab1) 5
[OK] один общий счетчик на Eco.Lab2, Eco.Lab1 и B
[OK] после Release счетчик вернулся к исходному
```

В листинге таблицы многочлена оставлена часть строк, программа печатает значения для x от -3 до 4.

Блок 5 в двух вариантах сборки:

```
EcoLab2:                                        EcoLab2C:
5. Компонент с CID_EcoCalculatorA: A или C      5. Компонент с CID_EcoCalculatorA: A
или C
[OK] по CID_EcoCalculatorA получен                [OK] по CID_EcoCalculatorA получен
     IEcoCalculatorX                                   IEcoCalculatorX
подключен компонент A: есть только X            подключен компонент C: есть X и Y, 6 *
7 = 42
[OK] компонент A: Y не поддерживается           [OK] компонент C: интерфейс Y работает
[OK] сложение через этот компонент              [OK] сложение через этот компонент
[OK] Eco.Lab2 работает без изменений            [OK] Eco.Lab2 работает без изменений

Итого: пройдено 20, провалено 0                 Итого: пройдено 20, провалено 0
Release последней ссылки на Eco.Lab2: 0         Release последней ссылки на Eco.Lab2: 0
```

Последний `Release` возвращает 0 в обоих случаях: все полученные интерфейсы освобождены, утечек ссылок нет.

8. Анализ

8.1. Сравнение приемов

Таблица 4. Включение и агрегирование

	Включение	**Агрегирование**
Кто реализует интерфейс	внешний компонент, своя таблица функций	внутренний компонент
Можно добавить свой код	да, до и после вызова	нет
Требования к внутреннему компоненту	никаких	должен поддерживать агрегирование
Стоимость вызова	лишний вызов через таблицу внешнего компонента	без накладных расходов
Объем кода во внешнем компоненте	по функции на каждый метод интерфейса	одна ветка в QueryInterface
Сложность	низкая	выше: два IEcoUnknown, правильный подсчет ссылок
Где применено в работе	Y в Eco.Lab1 (E), A и D в Eco.Lab2	X в Eco.Lab1 (B), Eco.Lab1 в Eco.Lab2

Включение подходит, когда внешний компонент должен контролировать интерфейс: проверять аргументы, менять поведение или использовать внутренний компонент только для своих вычислений. В работе это проверка деления на ноль в Eco.Lab1 и метод `Horner` в Eco.Lab2. Включать можно любой компонент, поэтому A, D и E используются только так.

Агрегирование подходит, когда интерфейс внутреннего компонента нужно отдать клиенту без изменений. Внешнему компоненту не нужно писать обертку для каждого метода, поэтому через Eco.Lab2 доступны все шесть методов `IEcoAdvancedMath` без единой строки кода, которая бы их повторяла. Цена в том, что внутренний компонент нужно писать специально: в этой работе пришлось переделать Eco.Lab1, а из пяти калькуляторов агрегирование поддерживает только B.

8.2. Подсчет ссылок и время жизни

Самое тонкое место агрегирования - счетчик ссылок. У агрегированного компонента нет своего видимого счетчика: все `AddRef` и `Release` его интерфейсов меняют счетчик внешнего компонента. Внутренний компонент живет ровно столько, сколько внешний, и удаляется в функции удаления внешнего через неделегирующий `Release`. Тесты 19 в EcoLab1 и 4 в EcoLab2 показывают это напрямую: `AddRef` у X компонента B увеличивает счетчик Eco.Lab1 или Eco.Lab2, а после освобождения всех указателей счетчик возвращается к исходному значению.

Ошибки, которые здесь легко сделать. Каждую из них тесты бы обнаружили: неверным указателем, неверным счетчиком или аварийным завершением.

- внутренний компонент получает не управляющий `IEcoUnknown`, а неделегирующий внешнего. Тогда при двух уровнях агрегирования `QueryInterface` у X компонента B не найдет `IEcoLab2`;
- фабрика запрашивает интерфейс у нового экземпляра через делегирующий `QueryInterface`. При агрегировании вызов уйдет во внешний компонент, который еще не создан;
- счетчик меняется и в делегирующих, и в неделегирующих функциях. Тогда при агрегировании ссылки считаются дважды и компонент не удаляется;
- внешний компонент освобождает внутренний через делегирующий `Release`. Вызов вернется в сам внешний компонент, и получится повторное удаление.

8.3. Устройство компонентов из курса

При изучении исходных текстов калькуляторов видно, что D и E решают одну задачу, получить X из B, разными способами. D включает B и реализует X заново, поэтому через D можно получить X, но это будет интерфейс самого D. E агрегирует B, поэтому через E клиент получает X компонента B. Тест 20 показывает, что оба способа работают одинаково с точки зрения клиента. Компонент D к тому же устойчив к отсутствию B: если B не зарегистрирован, он берет A. В Eco.Lab1 тот же прием использован для интерфейса Y: если нет E, берется D.

9. Сборка

Работа собиралась и проверялась под Linux (Ubuntu 24.04 в WSL2): gcc 13.2.0, make 4.4.1, Eco.Framework от 18.09.2026. Компоненты собираются в статические библиотеки. Порядок сборки: сначала внутренние компоненты, потом внешние.

```
for d in Eco.CalculatorA Eco.CalculatorB Eco.CalculatorC Eco.CalculatorD
Eco.CalculatorE \
         Eco.Lab1DP Eco.Lab1 Eco.Lab2; do
    (cd $d/AssemblyFiles/Linux/gcc_v132 && make -f Makefile TARGET=1 DEBUG=0
ARCH=x86_64)
done
```

Клиенты собираются файлами `MakefileExe` в папках Eco.Lab1 и Eco.Lab2. Файлы сборки для Windows (gcc и MSVC), macOS (Xcode) и EcoOS дополнены путями к калькуляторам, Eco.Lab1 и их библиотеками. Под Windows все компоненты компилируются компилятором MinGW без ошибок и предупреждений. Собрать исполняемый файл им нельзя, так как библиотеки

Eco.Framework для Windows рассчитаны на компилятор MSVC. Сборку в MSVC и Xcode я не проверял.

10. Выводы

- В компонент Eco.Lab1 добавлены интерфейсы калькулятора: `IEcoCalculatorX` через агрегирование компонента B и `IEcoCalculatorY` через включение компонента E (или D). Интерфейс первой работы `IEcoAdvancedMath` не изменился, все ее проверки проходят.
- Eco.Lab1 поддерживает агрегирование: у него есть делегирующий и неделегирующий `IEcoUnknown`.
- Создан компонент Eco.Lab2, который агрегирует Eco.Lab1 и включает A и D. Через Eco.Lab2 доступны интерфейсы Eco.Lab1, в том числе X компонента B через два уровня агрегирования.
- Задействованы все компоненты A, B, C, D, E и Eco.Lab1. Компонент C показан как замена A с тем же идентификатором без изменения кода клиента.
- Тесты подтверждают все правила `QueryInterface` и общий счетчик ссылок у собственных и агрегированных интерфейсов: всего 83 проверки, ошибок нет, утечек ссылок нет.
- Включение проще и позволяет менять поведение интерфейса, агрегирование избавляет от оберток, но требует специально подготовленного внутреннего компонента и аккуратного подсчета ссылок.

11. Источники

1. Башев В. Н. Проектирование ОС, технологии, инструменты. Лекции 2-4: технология АОМК, интерфейс IEcoUnknown, повторная применяемость, фабрика.

2. Задание 2 к лабораторной работе: включение и агрегирование компонентов калькулятора.

3. Учебный курс InsideACOM, уроки 2-7: компоненты Eco.CalculatorA - Eco.CalculatorE. https://github.com/peerf-eco/Eco.Education